OOP in C++

File Handling

Instructor: Dinesh Maharjan

2022

Unformatted IO

IO byte by byte.
High speed
Good for high volume.
Inconvenient for human.

```
char ch=cin.get();
cout.put(ch);

getline(cin,name);
```

IO is done by using meaningful groups sucha as int, char, long.
Good for IO
But not for high volume.
Formatted functions are setw, left, right, setprecision.

```
cin>a;
cout<<a;
cout<<setpricision<<a<<endl;
```

Formatted IO

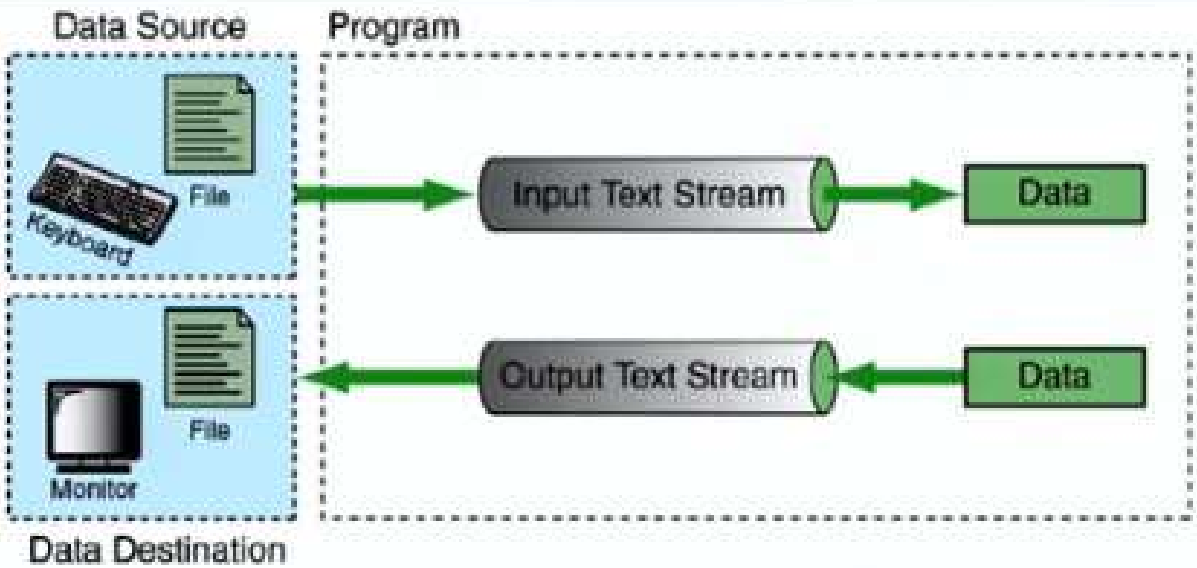
What is a file?

- A named collection of data, stored in secondary storage (typically).
- Typical operations on files:
  - Open
  - Read
  - Write
  - Close
- How is a file stored?
  - Stored as sequence of bytes, logically contiguous (may not be physically contiguous on disk).

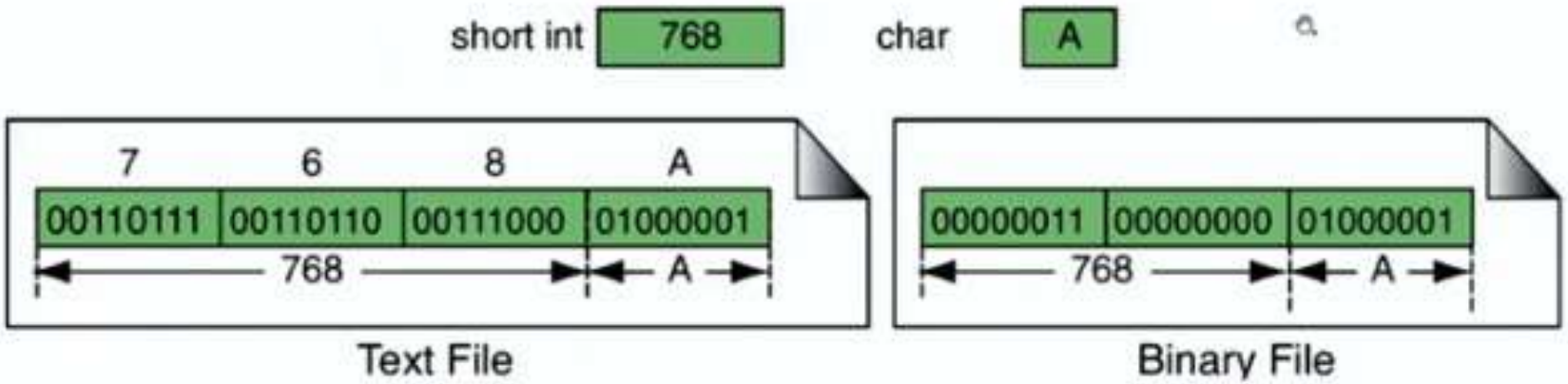
- The last byte of a file contains the end-of-file character (EOF), with ASCII code 1A (hex).
- While reading a text file, the EOF character can be checked to know the end.

- **Two kinds of files:**
  - Text :: contains ASCII codes only
  - Binary :: can contain non-ASCII characters
    - Image, audio, video, executable, etc.
    - To check the end of file here, the *file size* value (also stored on disk) needs to be checked.

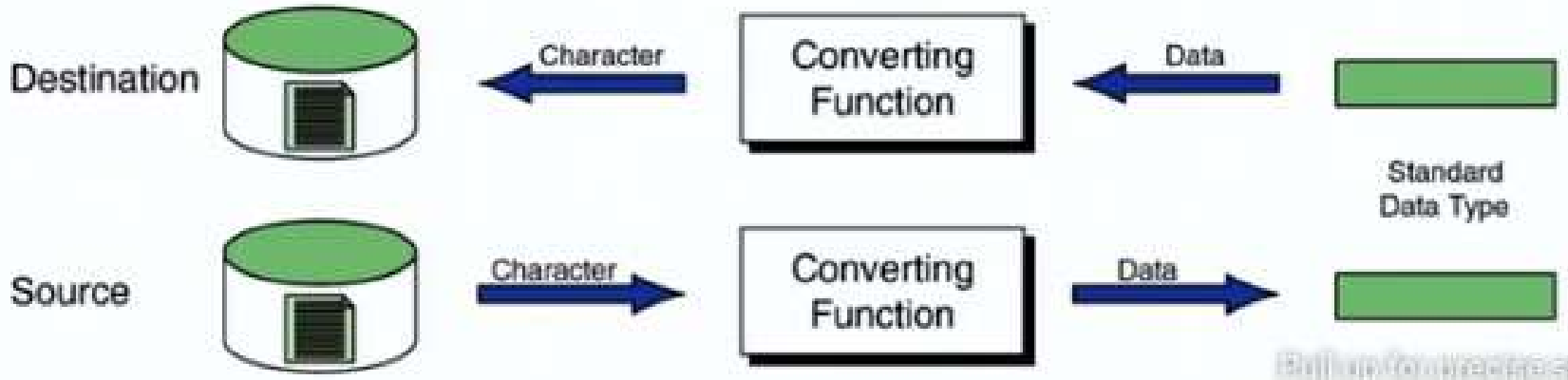
- *A* **file** *is an external collection of related data treated as a unit.* The primary purpose of a file is to keep a record of data. Since the contents of primary memory are lost when the computer is shut down, we need files to store our data in a more permanent form.
- *Data is input to and output from a* **stream**. A stream can be associated with a physical device, such as a terminal, or with a file stored in auxiliary memory.



- ascii(7) = 55 = 0b 0011 0111
- ascii(6) = 54 = 0b 0011 0110
- ascii(8) = 56 = 0b 0011 1000
- ascii('A') = 65 = 0b 0100 0001
- 768 = 0b 0000 0011 0000 0000



- Text files store data as a sequence of characters
- Binary files store data as they are stored in primary memory



Text files are used for:

- Formatted input/output functions
- Character input/output functions
- String input/output functions

33:08

Why to use Files:

- Convenient way to deal large quantities of data.
- Store data permanently (until file is deleted).
- Avoid typing data into program multiple times.
- Share data between programs.

We need to know:

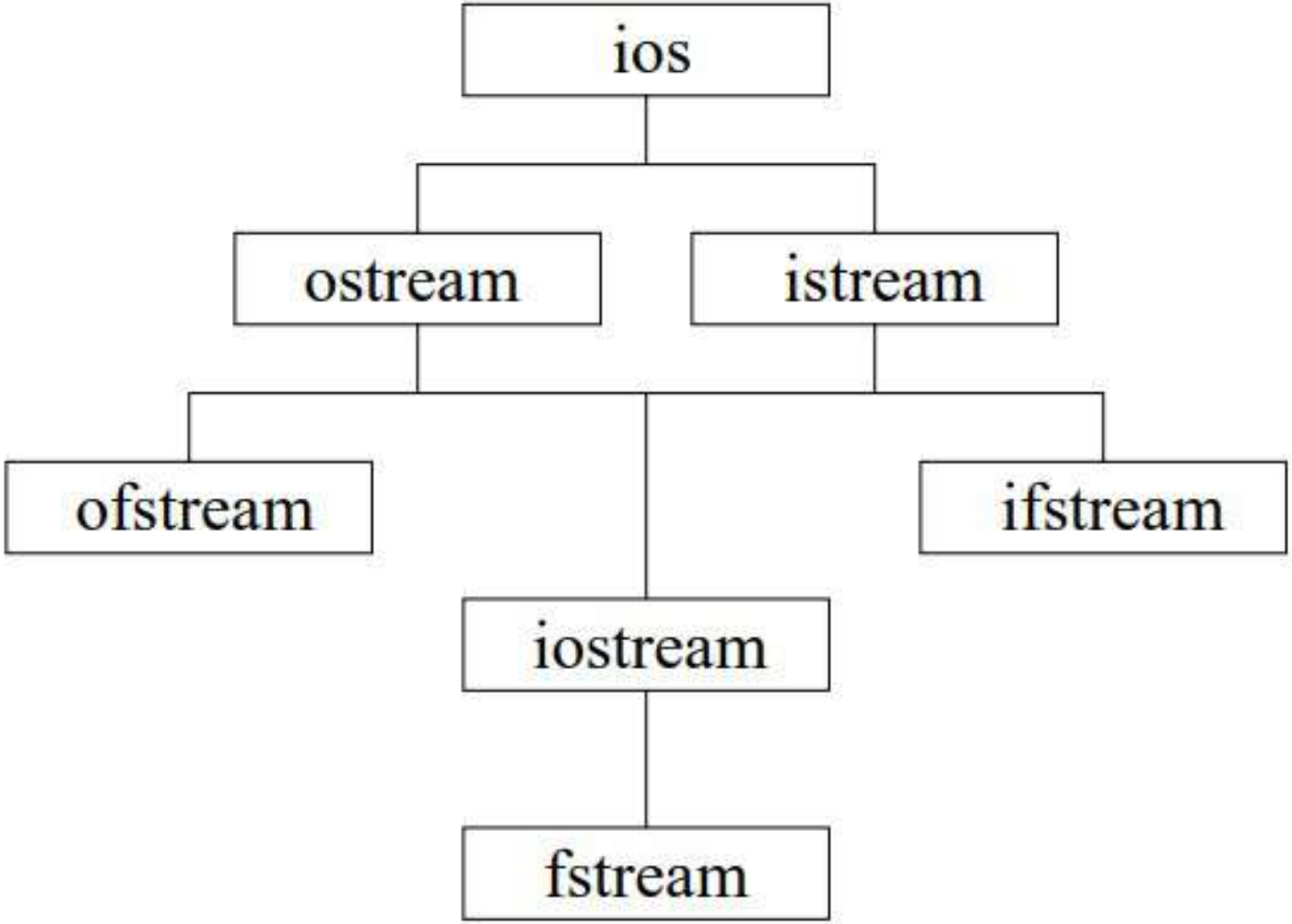
how to "connect" file to program

how to tell the program to read data

how to tell the program to write data

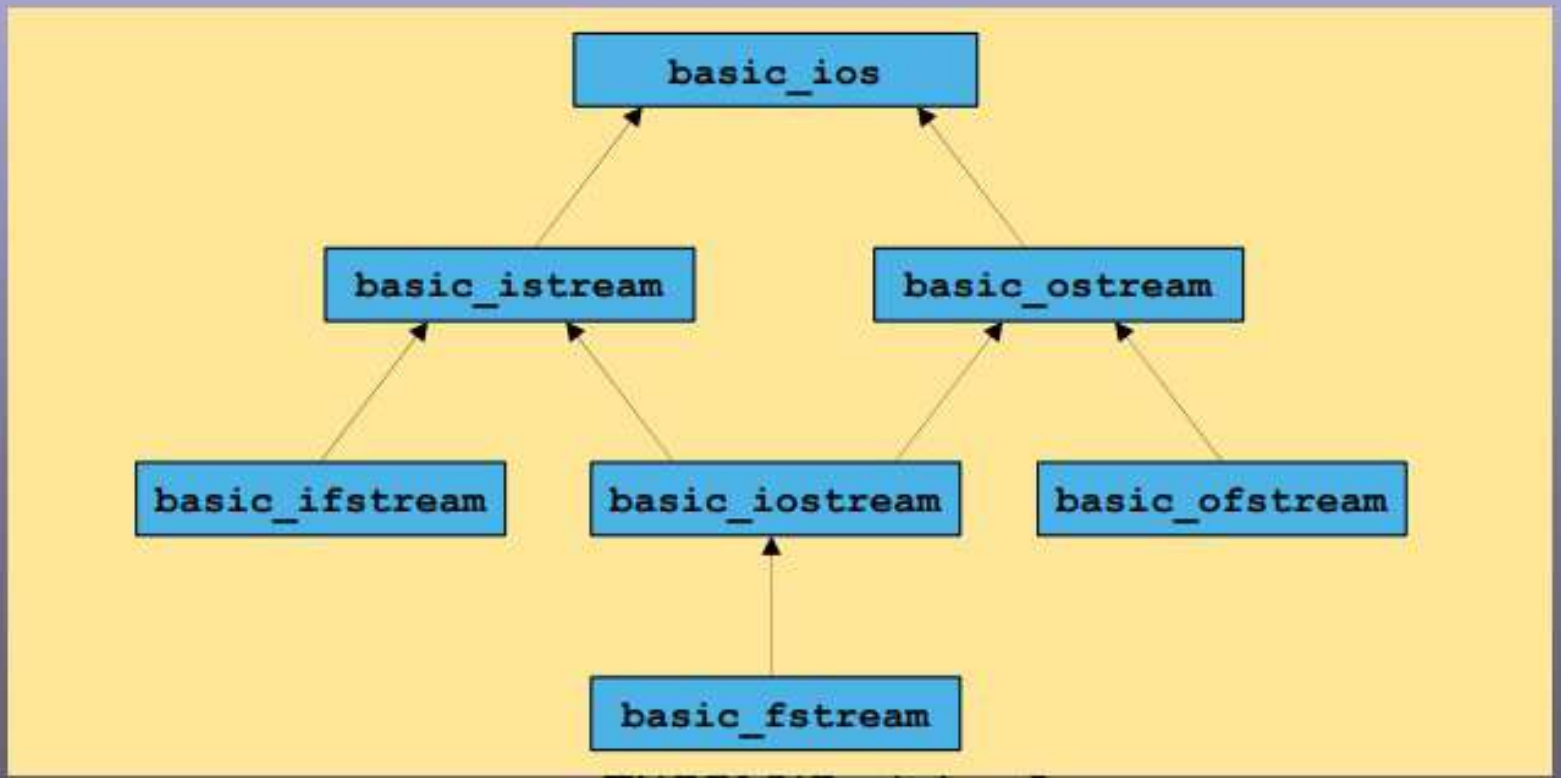
error checking and handling EOF

Stream Classes

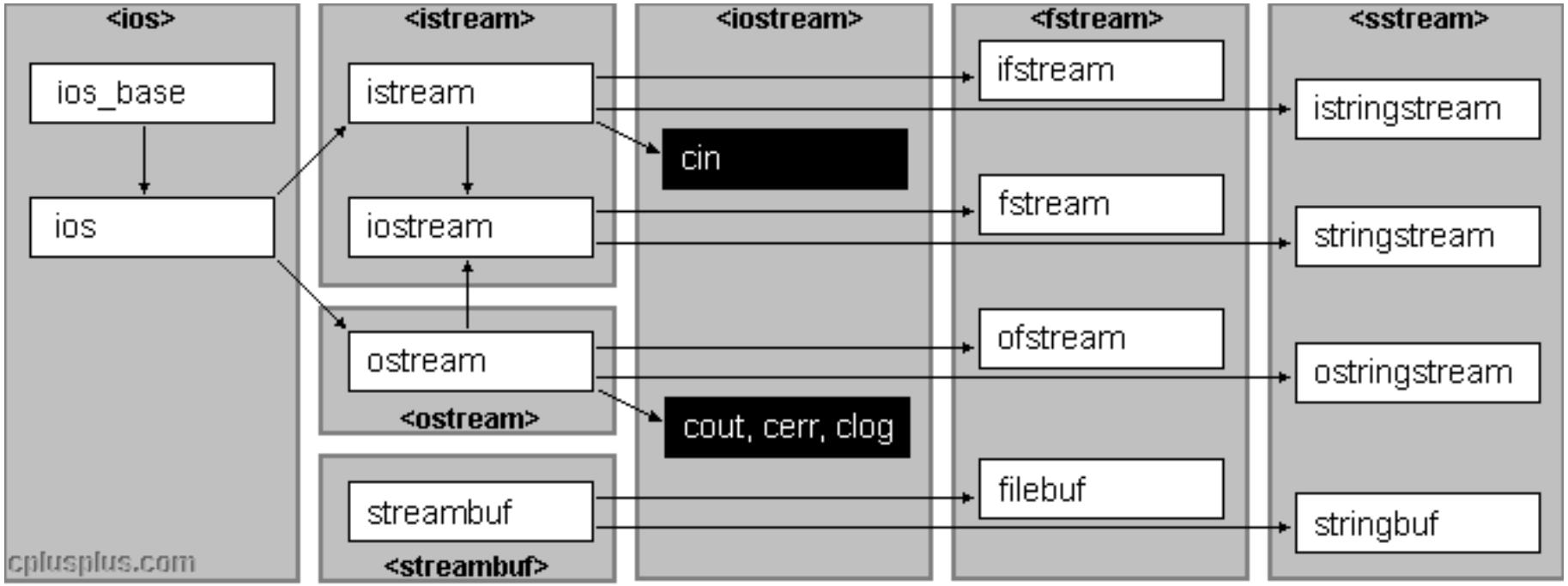


Stream Classes

- **`istream`** specialization of **`basic_istream`** (**`char`** input)
- **`ostream`** specialization of **`basic_ostream`** (**`char`** output)
- **`cin`** is an instance of **`istream`**
- **`cout`** is an instance of **`ostream`**



Stream Classes



Stream Classes

- ios is mother of all stream classes.
- In recent compilers, ios has been replaced by ios_base.
- Ios keeps track of error state
- It converts ASCII characters into integer, double and vice versa
- Cout is object of ostream.
- Cin is object of istream.
- Object of ifstream deal with input files.
- Objects of ofstream deal with writing files.
- Fstream deal with reading and writing files
- Ofstream, ifstream, fstream are contained in fstream.h

Opening & Closing a File

- A file can be open by the method "open()" or immediately in the constructor (the natural and preferred way).

void ofstream / ifstream::open(const char* filename, int mode);

- filename - file to open (full path or local)
- mode - how to open (1 or more of following - using |)
  - ios::app - append
  - ios::ate - open with marker at the end of the file
  - ios::in / ios::out - (the defaults of ifstream and ofstream)
  - ios:nocreate / ios::noreplace - open only if the file exists / doesn't exist
  - ios::trunc - open an empty file
  - ios::binary - open a binary file (default is textual)
- Don't forget to close the file using the method "close()"

1: To access file handling routines:

```
#include <fstream.h>
```

2: To declare variables that can be used to access file:

```
ifstream in_stream;
ofstream out_stream;
```

3: To connect your program's variable (its internal name) to an external file (i.e., on the Unix file system):

```
in_stream.open("infile.dat");
out_stream.open("outfile.dat");
```

4: To see if the file opened successfully:

```
if (in_stream.fail())
{    cout << "Input file open failed\n";
     exit(1);     // requires <stdlib.h>}
```

Open with

5: To get data from a file (one option), must declare a variable to hold the data and then read it using the ***extraction*** *operator:*

int num;

in_stream >> num;

[Compare: cin >> num;]

6: To put data into a file, use insertion operator:

out_stream << num;

[Compare: cout << num;]

NOTE: Streams are sequential – data is read and written in order – generally can't back up.

7: When done with the file:

in_stream.close();

out_stream.close();

Creating a Sequential-Access File

- To open file
  - Create a stream object
    - Creates "line of communication" from object to file
  - Classes
    - **`ifstream`** (input only)
    - **`ofstream`** (output only)
    - **`fstream`** (I/O)
  - Constructors take *file name* and *file-open mode*

```
ofstream outClientFile( "filename", fileOpenMode );
```

  - To attach a file later

```
ofstream outClientFile;
outClientFile.open( "filename", fileOpenMode);
```

Creating a Sequential-Access File

- File-open modes

Mode	Description
ios::app	Write all output to the end of the file.
ios::ate	Open a file for output and move to the end of the file (normally used to append data to a file). Data can be written anywhere in the file.
ios::in	Open a file for input.
ios::out	Open a file for output.
ios::trunc	Discard the file's contents if it exists (this is also the default action for **ios::out**)
ios::binary	Open a file for binary (i.e., non-text) input or output.

  - **ofstream** opened for output by default

```
ofstream outClientFile( "clients.dat", ios::out );
ofstream outClientFile( "clients.dat");
fstream ClientFile( "clients.dat", ios::out | ios::in);
```

File Operations

- Writing to file (just like **`cout`**)
  - **`outClientFile << myVariable`**
- Closing file
  - **`outClientFile.close()`**
  - Automatically closed when destructor called

Open a File

```
#include <iostream>
#include <fstream>
```

Notice the the header files required for file I/O.

```
//ofstream constructor opens file
 ofstream moviesData( "reviews.txt", ios::out );

 //exit program if unable to create file
 if ( !moviesData ) {
      cerr << "File could not be opened" << endl;
      return;
  }
```

Writing to File

```
#include<iostream>
#include<string>
#include<fstream>
using namespace std;
int main(){
        int roll;
        string name,address;
        ofstream fout;
        fout.open("student.txt");
        if(!fout){
                cout<<"File could not be opened"<<endl;
                exit(1);
        }
        int choice=1;
        while(choice){
                cout<<"Enter name, roll and address"<<endl;
                cin>>name>>roll>>address;
                fout<<name<<" "<<roll<<" "<<address<<endl;
                cout<<"Enter 1 to add more otherwise 0"<<endl;
                cin>>choice;
        }
        fout.close();
        return 0;
}
```

Appending to the File

```
#include<iostream>
#include<string>
#include<fstream>
using namespace std;
int main(){
        int roll;
        string name,address;
        ofstream fout;
        fout.open("student.txt",ios::app);
        if(!fout){
                cout<<"File could not be opened"<<endl;
                exit(1);
        }
        int choice=1;
        while(choice){
                cout<<"Enter name, roll and address"<<endl;
                cin>>name>>roll>>address;
                fout<<name<<" "<<roll<<" "<<address<<endl;
                cout<<"Enter 1 to add more otherwise 0"<<endl;
                cin>>choice;
        }
        fout.close();
        return 0;
}
```

Read Movies Data File

- Assume that the file looks as follows

Mystic River: 4
Lost: 1
The Reader: 5
Pulp Fiction: 5
No Country for Old Man: 5
...

The loop is execute once, if the file is empty
Doesn't work!!!

```
while ( !moviesFile.eof() ) {
    moviesFile.getline(name, 40, ':');
    moviesFile >> score;
    cout << name << ": " << score << endl;
    moviesFile.ignore();
}
```

Testing with `eof()`

- `moviesFile.eof()`
  - First read then test if end-of-file
  - If `EOF` is reached
    - `getline()` and `>>` give no error
    - `eofbit` is set
    - `eof()` returns `true`

Reading Files

Ifstream obj;
Obj>>data>>

- It reads characters till space or end of line
- However, it does not store space, tab or end of line.
- It does not produce any error while reading end of file.

Reading Files

Ifstream obj;
Obj>>data>>

- Reading with object does not remove the non-delimiter characters from the stream after reading the text.
- If non-delimiter characters appear in the beginning of stream, then it will remove those characters from the stream.
-
- However, getline will read entire line including new line character.
- New line character will not be stored in the variable.
- So, its good to use getline function will counting the number of lines.

Reading from File

```
#include<iostream>
#include<string>
#include<fstream>
using namespace std;
int main(){
        string name,address;
        int roll;
        ifstream fin;
        fin.open("student.txt",ios::in);
        if(!fin){
                cout<<"File could not be opened"<<endl;
                exit(1);
        }
        cout<<"Name:"<<" "<<"Roll"<<" "<<"Address:"<<endl;
        while(!fin.eof()){
                fin>>name>>roll>>address;
                cout<<name<<” “<<roll<<” “ <<address<<endl;
        }
        fin.close();
        return 0;
}
```

Reading from File

/home/dnes/cPlusPlus1/NewLab/SetDemo

```
dan 1 eagan
pat 2 melborn
pat 2 melborn

Process returned 0 (0x0)   execution time : 0.003 s
Press ENTER to continue.
```

Reading from File

```
#include<iostream>
#include<string>
#include<fstream>
using namespace std;
int main(){
        string line;
        ifstream fin;
        fin.open("student.txt",ios::in);
        if(!fin){
                cout<<"File could not be opened"<<endl;
                exit(1);
        }
        cout<<"Name:"<<" "<<"Roll"<<" "<<"Address:"<<endl;
        while(!fin.eof()){
                getline(fin,line);
                cout<<line<<endl;
        }
        fin.close();
        return 0;
}
```

Reading from File

```
/home/dnes/cPlusPlus1/NewLab/SetDemo

dan 1 eagan
pat 2 melborn


Process returned 0 (0x0)   execution time : 0.004 s
Press ENTER to continue.
```

Copying content of file

```
int main(){
        string text;
        ofstream fout("studentCopy.txt");
        ifstream fin("student.txt");
        if(!fout || !fin){
    cout<<"File could not be opened"<<endl;
    exit(1);
        }
        while(!fin.eof()){
    getline(fin,text);
    fout<<text<<endl;
        }
        fin.close();
        fout.close();
        return 0;
}
```

Counting number of lines

```
int main(){
        string text;
        int count=0;
        ifstream fin("student.txt");
        if(!fin){
    cout<<"File could not be opened"<<endl;
    exit(1);
        }
        while(!fin.eof()){
    getline(fin,text);
   count++;
        }
        cout<<”Number of lines=”<<count<<endl;
        fin.close();
        return 0;
}
```

File Position Pointers

- Number of next byte to read/write
- Functions to reposition pointer
  - **`seekg`** (seek get for **`istream`** class)
  - **`seekp`** (seek put for **`ostream`** class)
  - Classes have "get" and "put" pointers
- **`seekg`** and **`seekp`** take *offset* and *reference position*
  - Offset: number of bytes relative to a reference position
  - Reference positions (**`ios::beg`** default)
    - **`ios::beg`** - relative to beginning of stream
    - **`ios::cur`** - relative to current position
    - **`ios::end`** - relative to end

File Position Pointers

- Examples
  - **`fileObject.seekg(0)`**
    - Goes to front of file (location **`0`**) because **`ios::beg`** is default
  - **`fileObject.seekg(n)`**
    - Goes to **`n`**th byte from beginning
  - **`fileObject.seekg(n, ios::cur)`**
    - Goes **`n`** bytes forward
  - **`fileObject.seekg(n, ios::end)`**
    - Goes **`n`** bytes back from end
  - **`fileObject.seekg(0, ios::end)`**
    - Goes to last byte
  - **`seekp`** similar

File Position Pointers

- To find pointer location
  - **tellg** and **tellp**

```
long location;
location = fileObject.tellg()
```

Example

- Assume you need to scan a file twice from the beginning

```
//read file from beginning to EOF
…
// reset eofbit for next input
file.clear();
// move to beginning of file
file.seekg( 0 );
//read again file from beginning to EOF
```

Updating Sequential-Access Files

- Updating sequential files
  - Risk overwriting other data
  - Example: change name "White" to "Worthington"
    - Old data

```
300 White 0.00 400 Jones 32.87
```

    - Insert new data

```
300 Worthington 0.00
```

↓

```
300 White 0.00 400 Jones 32.87
```

↓

Data gets overwritten

```
300 Worthington 0.00ones 32.87
```

  - Formatted text different from internal representation

Random-Access Files

- Instant access
  - Want to locate record quickly
    - Airline reservations, ATMs
  - Sequential files must search through each one
- Random-access files are solution
  - Instant access
  - Insert record without destroying other data
    - As if an empty record was previously created
  - Update/delete items without changing other data

Writing Randomly

```
int main(){
        ofstream out("student1.txt");
        if(!out){
       cout<<"File could not be opened"<<endl;
       exit(1);
        }
        cout<<out.tellp()<<endl; //0
        out<<"Hello World";
        out.seekp(6);
        cout<<out.tellp()<<endl; //6
        out<<"Nepal"; // Hello Nepal is saved
        out.close();
        return 0;
}
```

Reading Randomly

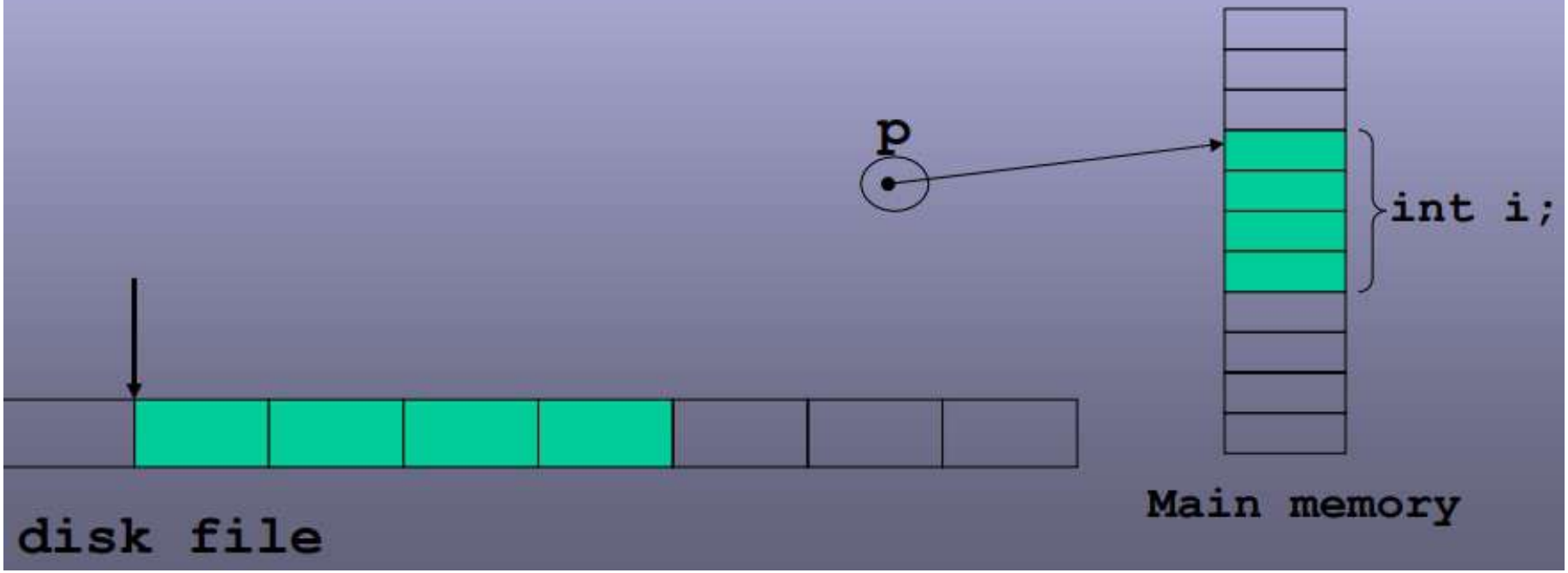
```
int main(){
        string name;
        ifstream reader("student1.txt",ios::in);
        if(!reader){
                cout<<"File could not be opened"<<endl;
               exit(1);
        }
        cout<<reader.tellg()<<endl;  //0
        reader.seekg(6);
        while(!reader.eof()){
       reader>>name;
       cout<<name<<endl;  //Nepal
        }
        reader.close();
        return 0;
}
```

Random-Access File vs Text File

- **`"1234567"`** (**`char*`**) vs **`1234567`** (**`int`**)
  - **`char*`** takes 8 bytes (1 for each character + '\0')
  - **`int`** takes fixed number of bytes (perhaps 4)
    - 123 same size in bytes as 1234567
- **`<<`** operator and **`write()`**
  - **`outFile << number`**
    - Outputs **`number`** (**`int`**) as a **`char*`**
    - Variable number of bytes
  - **`outFile.write( const char*, int );`**
    - Outputs raw bytes
    - Takes pointer to memory location and number of bytes to write
      - Copies data directly from memory into file

file.write(const char*, int);

```
int i = 5567;
int*p = &i;
file.seekp(8);
file.write( ( char* ) p , sizeof( int ) );
```



Creating a Random-Access File

- Example

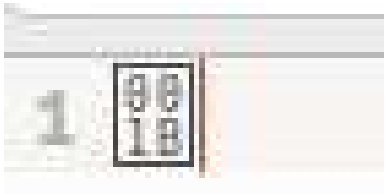
```
outFile.write( (char*)( &number ), sizeof( number ) );
```

  - **&number** is an **int***
    - Convert to **char***
  - **sizeof(number)**
    - Size of **number** (an **int**) in bytes
  - **read** function similar (more later)

Write Binary

```
#include<iostream>
#include<fstream>
using namespace std;

int main(){
        // 27 = 11011
        int a=27, *p;
        ofstream writer("binaryFile1.txt",ios::binary);
        if(writer.fail()){
        cout<<"File could not be opened";
        exit(1);
        }
        p=&a;
        writer.write((char *)p,1); //write one byte file size 1 Byte
        // text file takes 2 bytes
        writer.close();
        return 0;
}
```



Reading Data Sequentially from a Binary File

- **`read`** - similar to **`write`**
  - Reads raw bytes from file into memory
  - `inFile.read( (char*)( &number ), sizeof( int ) );`
    - **`&number`**: location of the data
    - **`sizeof(int)`** : how many bytes to read
  - Do not use **`inFile >> number`** with raw bytes
    - **`>>`** converts **`number`** to text format
    - `e.g. 123` does not occupy same number of bytes as `1234`

Read Binary

```
#include<iostream>
#include<fstream>
using namespace std;

int main(){
        int a,*p;
        ifstream reader("binaryFile2.txt",ios::binary);
        if(reader.fail()){
        cout<<"File could not be opened";
        exit(1);
        }
        p=&a;
        reader.read((char *)p,1);
        cout<<*p<<endl;  //27
        reader.close();
        return 0;
}
```